

You are not tied to having a single configuration file, and, in fact, there are specific directives that can be used to include other configuration files. The configuration of the cluster will be detailed in a later chapter.

The PostgreSQL HBA file (`pg_hba.conf`) is another text file that contains the connection allowance: it lists the databases, users, and networks that are allowed to connect to your cluster. The HBA method can be thought of as a *firewall* embedded into PostgreSQL. As an example, the following is an excerpt from a `pg_hba.conf` file:

```
hosts    all  luca  192.168.222.1/32  md5
hostssl  all  enrico 192.168.222.1/32  md5
```

In short, the preceding lines mean that the user `luca` can connect to any database in the cluster with the machine with the IPv4 address `192.168.222.1`, while the user `enrico` can connect to any database from the same machine but only on an SSL-encrypted connection. All the available `pg_hba.conf` rules will be detailed in a later chapter, but for now, it is sufficient to know that this file acts as a “list of firewall rules” for incoming connections.

Summary

PostgreSQL can handle several databases within a single cluster, served out of disk storage contained in a single directory named `PGDATA`. The cluster runs many different processes; one, in particular, is named `postmaster` and is in charge of spawning other processes, one per client connection, and keeping track of the status of maintenance processes.

The configuration of the cluster is managed via text-based configuration files, the main one being `postgresql.conf`. It is possible to filter incoming user connections by means of rules placed in the `pg_hba.conf` text file.

You can interact with the cluster status by means of the `pg_ctl` tool or, depending on your operating system, by other provided programs, such as `service` or `systemctl`.

This chapter has presented you with the relevant information so that you are able not only to install PostgreSQL but also to start and stop it regularly, integrate it with your operating system, and connect to the cluster.

In the following chapter, you will learn how to manage users and connections.

Verify your knowledge

- What is the `pg_ctl` command?

`pg_ctl` is a command shipped with PostgreSQL that allows you to start, restart, stop, and do other actions on the cluster. It is often used as the way to manage the whole cluster. See the *pg_ctl* section for more details.

- What is a template database?

A template database is a database that can be used as a base to clone another (new) database that will initially include the same objects. See the *The template databases* section for more details.

- What is the `psql` command?

`psql` is the official client application to connect to a PostgreSQL database. It is a command - line application that can be used to enter SQL statements and get results out of the cluster. It is shipped with every version of PostgreSQL. See the *The psql command-line client* section for more details.

- What is a connection string?

A connection string is a URI that specifies all the properties required to connect to a database, often including the username, the host, the database, and so on. See the *The connection string* section for more details.

- What are the `psql` special commands?

The special commands are all the short commands that begin with a backslash symbol, like, for example, `\d`. They are informative commands valid only within the `psql` client. See the *A glance at the psql commands* section for more details.

References

- PostgreSQL PGDATA disk layout: <https://www.postgresql.org/docs/current/storage-file-layout.html>
- PostgreSQL initdb official documentation: <https://www.postgresql.org/docs/current/app-initdb.html>

- PostgreSQL pg_ctl official documentation: <https://www.postgresql.org/docs/current/app-pg-ctl.html>
- The pgAdmin4 graphical client for PostgreSQL: <https://www.pgadmin.org/>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



3

Managing Users and Connections

PostgreSQL is a complex system that includes users, databases, and data. In order to be able to interact with a database in the cluster, you need to have at least one user. By default, when installing a new cluster, a single administrator user (named `postgres`) is created. While it is possible to handle all the connections, applications, and databases with that single administrative user, it is much better for security and privilege isolation to create different users with different properties and privileges, as well as login credentials, for every specific task.

PostgreSQL provides a very rich user-management structure, and single users can be grouped into a variety of different groups at the same time. Moreover, groups can be nested within other groups so that you can have a very accurate representation of your account model. Thanks to this accurate representation, and thanks to the fact that every user and group can be assigned different properties and privileges, it is possible to apply fine-grained permissions to each user in the database, depending on the specific task and activity involved.

This chapter introduces you to the concepts behind users and groups and their relationships. The chapter will focus mainly on the login properties of roles (either users or groups) and how PostgreSQL can prevent specific users from connecting to specific databases.

This chapter covers the following main topics:

- Introduction to users and groups
- Managing roles
- Managing incoming connections at the role level

Technical requirements

The chapter examples can be run on the standalone Docker image that you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>. For installation and usage instructions of the Docker images for this book, please refer to *Chapter 1, Introduction to PostgreSQL*.

Introduction to users and groups

PostgreSQL distinguishes between *users* and *groups of users*: the former represents someone, either a person or an application, that could connect to the cluster and perform activities; the latter represents a collection of users that share some common properties, most commonly permissions on cluster objects.

In order to connect interactively or via an application to a PostgreSQL database, you need to have login credentials. In particular, a database user, a user who is allowed to connect to that specific database, must exist.

Database users are somewhat similar to operating system users: they have a username and an (encrypted) password and are known to the PostgreSQL cluster. Similarly to operating system users, database users can be grouped into **user groups** in order to make their management easier.

In SQL, and therefore also in PostgreSQL, the concepts of both a single user account and a group of accounts are encompassed by the concept of a **role**.

A role can be a single account, a group of accounts, or even both depending on how you design it; however, in order to make management easier, a role should express one and only one concept at a time: that is, it should be either a single user or a single group, but not both.



While a role can be used simultaneously as a group or a single user, we strongly encourage you to keep the two concepts of user and group separate—it will simplify the management of your infrastructure.

Every role must have a unique name or identifier, usually called a username.

A role represents a collection of database permissions and connection properties. The two elements are orthogonal. You can set up a role simply as a container for other roles, configuring the contained roles to hold the assigned permissions, or you can have a role that holds all the permissions for contained roles, or mix and match these two approaches.

It is important to understand that a role is defined at the cluster level, while permissions are defined at the database level. This means that the same role can have different privileges and properties depending on the database it is using (for instance, being allowed to connect to one database and not to another).



Since a role is defined at the cluster level, it must have a unique name within the entire cluster.

Managing roles

Roles can be managed by means of three main SQL statements: `CREATE ROLE` to create a role from scratch, `ALTER ROLE` to change some role properties (for example, the login password), and `DROP ROLE` to remove an existing role.



PostgreSQL ships with operating system tools to manage roles: `createuser` and `dropuser`. Both these commands open a connection to the cluster and perform the SQL commands mentioned above; therefore, the usage of these tools will not be explained in this chapter.

In order to use the SQL statements to create new roles and then manage them, it is necessary to connect to a database in the cluster. The superuser role `postgres` can be used to that aim, at least initially, since such a role is created when the database cluster is initialized. Using the `postgres` role and a template database is the most common way to create your initial roles.

A role is identified by a string that represents the role name, or better, the account name of that role. This name must be unique across the system, meaning that you cannot have two different roles with identical names. Names must consist of letters, digits, and some symbols, such as underscores.

Creating new roles

In order to create a new role, either a single user account or a group container, you need to use the `CREATE ROLE` statement. The statement has the following short synopsis and has a mandatory parameter, which is the role's username:

```
CREATE ROLE name [ [ WITH ] option [ ... ] ]
```

The options that you can specify in the statement range from the account password, the ability to log in interactively, and the superuser privileges. Please remember that, unlike other systems, in PostgreSQL, you can have as many superusers as you want, and everyone has the same live-or-die rights on the cluster.

Almost every option of the `CREATE ROLE` statement has a positive form that adds the ability to the role, and a negative form (with a `NO` prefix) that excludes the ability from the role. As an example, the `SUPERUSER` option adds the ability to act as a cluster superuser, while the `NOSUPERUSER` option removes it from the role.

In this chapter, we will focus on the login abilities, which is a restricted set of options that allows a role to log in to the cluster. Other options will be discussed in *Chapter 10, Users, Roles, and Database Security*, since they are more related to the security features of the role.



What if you forgot an option at the `CREATE ROLE` time? And what if you changed your mind and wanted to remove an option from an existing role? There is an `ALTER ROLE` statement that allows you (as a cluster superuser) to modify an existing role without having to drop and recreate it. The statement will be shown in *Chapter 10, Users, Roles, and Database Security*, along with some other interesting options for roles.

Role passwords, connections, and availability

Every connection to PostgreSQL must be made to a specific database, no matter the user that is opening the connection. Connecting to a database in the cluster means that the role must authenticate itself, and therefore, there must be an authentication mechanism, the username and password being the most classical ones.

When a user attempts to connect to a database, PostgreSQL checks the login credentials and a few other properties of the user to ensure that it is allowed to log in and has valid credentials.

The main options that allow you to manipulate and manage the login attempts are as follows:

- `PASSWORD` or `ENCRYPTED PASSWORD` are equivalent options and allow you to set the login password for the role. Both options exist for backward compatibility with older PostgreSQL versions, but nowadays, the cluster always stores role passwords in an encrypted form, so the use of `ENCRYPTED PASSWORD` does not add any value to the `PASSWORD` option.
- `PASSWORD NULL` explicitly forces a null (not empty) password, preventing the user from logging in with any password. This option can be used to deny password-based authentication.

- `CONNECTION LIMIT <n>` allows the user to open no more than `<n>` simultaneous connections to the cluster, without any regard to a specific database. This is often useful to prevent a user from wasting resources on the cluster.
- `VALID UNTIL` allows you to specify an instant (in the future) when the role will expire.

Setting the password for a specific role does not mean that that role will be able to connect to the cluster: in order to be allowed to interactively log in, the role must also have the `LOGIN` option. In other words, the following statement will not allow the user to log in:

```
postgres=# CREATE ROLE luca
          WITH PASSWORD 'xxx';
```

The default option is `NOLOGIN` (which prevents interactive login). Therefore, in order to define interactive users, remember to add the `LOGIN` option when creating the role:

```
template1=# CREATE ROLE luca
          WITH LOGIN PASSWORD 'xxx';
```

Multiple options can be written in any order, so the preceding code represents the same statement, but in a form that is less human readable:

```
postgres=# CREATE ROLE luca
          WITH PASSWORD 'xxx' LOGIN;
```

The `VALID UNTIL` option allows you to define a date or even a timestamp (that is, an instant) in the future when the role password will expire and will no longer be allowed to log in to the cluster. This can be useful for marking a set of users as dismissable in the future.

Of course, this option only makes sense for interactive roles, that is, those who have the `LOGIN` capability. As an example, the following role will be prevented from logging in after Christmas 2030:

```
postgres=# CREATE ROLE luca
          WITH LOGIN PASSWORD 'xxx'
          VALID UNTIL '2030-12-25 23:59:59';
```

Using a role as a group

A group is a role that contains other roles. It's that simple!

Usually, when you want to create a group, all you need to do is create a role without the `LOGIN` option and then add all the members one after the other to the **containing role**. Adding a role to a containing role makes the latter a group.

In order to create a role as a member of a specific group, the `IN ROLE` option can be used. This option accepts the name of the group (which, in turn, is another role) to which the newly created role will become a member. As an example, in the following code block, you can see the creation of the `book_authors` group and the addition of the role members `luca` and `enrico`:

```
postgres=# CREATE ROLE book_authors
           WITH NOLOGIN;
CREATE ROLE
postgres=# CREATE ROLE luca
           WITH LOGIN PASSWORD 'xxx'
           IN ROLE book_authors;
CREATE ROLE
postgres=# CREATE ROLE enrico
           WITH LOGIN PASSWORD 'xxx'
           IN ROLE book_authors;
CREATE ROLE
```



The `IN GROUP` clause of `CREATE ROLE` is an obsolete synonym for the `IN ROLE` clause.

It is also possible to add members to a group using the special `GRANT` statement. The `GRANT` statement is the general SQL statement that allows fine privilege tuning (more on this in *Chapter 10, Users, Roles, and Database Security*); PostgreSQL extends the SQL syntax allowing the *granting of a role to another role*. When you grant a role to another, the latter becomes a member of the former. In other words, assuming that all roles already exist without any particular association, the following adds the role `enrico` to the `book_authors` group:

```
postgres=# GRANT book_authors TO enrico;
```

Every group can have one or more **admin** members, which are allowed to add new members to the group. The `ADMIN` option allows a user to specify the member that will be associated as an administrator of the newly created group. For instance, in the following code block, you can see the creation of the new group called `book_reviewers` with `luca` as administrator; this means that the user `luca`, even if he is not a cluster superuser, will be able to add new members to the `book_reviewers` group:

```
postgres=# CREATE ROLE book_reviewers
           WITH NOLOGIN
           ADMIN luca;

CREATE ROLE
```

Clearly, the `ADMIN` option can be used in `CREATE ROLE` only if the administrator role already exist; in the example, the `luca` role must have been created before the group, as he is going to be the administrator.

The `GRANT` statement can solve the problem—the `WITH ADMIN OPTION` clause allows the membership of a role with administrative privileges.

As an example, the following piece of code shows how to make the user `enrico` also an administrator of the `book_reviewers` group. Please note that you must spell out `WITH ADMIN OPTION` in its entirety, as shown here:

```
postgres=# GRANT book_reviewers
           TO enrico
           WITH ADMIN OPTION;

GRANT ROLE
```

What happens if a group role has the `LOGIN` option? The group will still be a role container, but it can act also as a single user account with the ability to log in. While this is possible, it is a more common practice to deny group roles access to log in to prevent confusion.

Removing an existing role

In order to remove an existing role, you need to use the `DROP ROLE` statement. The statement has a very simple synopsis:

```
DROP ROLE [ IF EXISTS ] name [, ...]
```

You need to specify only the role name you want to delete, or, if you need to delete multiple roles, you can specify them as a comma-separated list.

In order to be deleted, the role must exist; therefore, if you try to remove a nonexistent role, you will receive an error:

```
postgres=# DROP ROLE this_role_does_not_exist;
ERROR:  role "this_role_does_not_exist" does not exist
```